From Tool-Using Language Models to Reliable Scientific Agents for Computational Chemistry

Wenxi Zhai[†] and Jinzhe Zeng[*,†]

†*School of Artificial Intelligence and Data Science, University of Science and Technology of China, Hefei 230026, P.R. China*

‡*Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215123, P.R. China*

¶*Shanghai Innovation Institute, Shanghai 200003, P.R. China*

§*Suzhou Big Data & AI Research and Engineering Center, Suzhou 215123, P.R. China*

E-mail: jinzhe.zeng@ustc.edu.cn

Abstract

Artificial intelligence (AI) is entering a stage in which models are embedded in tool-mediated systems that participate in real-world scientific processes. Yet computational chemistry presents a distinct challenge: successful command execution does not guarantee chemically meaningful, reproducible, or scientifically trustworthy results. In this perspective, we discuss the transition from tool-using language models to reliable scientific agents for computational chemistry and outline the technical and scientific requirements for verifiable agentic execution. We argue that three requirements are central to this transition. First, the growing complexity of computational chemistry software ecosystems demands skill-based agents that encapsulate reusable scientific procedures rather than merely invoking isolated tools. Second, computational chemistry requires agent frameworks that are more adaptive than conventional workflows,

yet more structured and accountable than unconstrained LLM agents. Third, because procedural knowledge in computational science is highly task-specific and context-dependent, agentic grounding is essential to anchor model reasoning in executable environments, runtime feedback, and verifiable scientific constraints. By reframing scientific agents as reliability-oriented execution systems, this perspective charts a path toward more rigorous, reproducible, and practically useful agentic computational chemistry.

1. Introduction

AI systems increasingly extend beyond standalone models through agentic architectures that connect language-based reasoning with external tools, scientific software, and executable environments. This development broadens the role of AI from generating isolated outputs to coordinating actions within operational frameworks. Recent surveys of LLM-based agents describe this paradigm in terms of tool use, planning, and feedback-driven adaptation.[1] In parallel, agent-skill frameworks treat reusable skills as modular carriers of procedural capability, enabling executable knowledge to be acquired, composed, and updated beyond a single prompt.[2,3] OpenClaw makes this direction concrete by organizing tasks through skills, execution contexts, and feedback from real environments.[4] Together, these developments signal that the next generation of AI agents will depend not only on stronger language understanding, but also on the ability to translate reasoning into structured, tool-mediated, and verifiable action.

This transition has rapidly reached chemistry and scientific research. Chemistry offers a natural proving ground: chemical discovery requires agents to connect literature knowledge, molecular reasoning, predictive models, synthesis planning, and executable tools. ChemCrow demonstrated that LLMs augmented with chemistry-specific tools can support organic synthesis, drug discovery, and materials design, showing how general language models can be transformed into chemistry-aware tool-using agents.[5] Recent reviews and perspectives

on autonomous agents in chemistry further indicate that the field is moving from isolated tool use toward integrated workflows that combine literature search, synthesis planning, automated experimentation, multimodal reasoning, and closed-loop optimization.[6,7] Beyond chemistry-specific tasks, Co-Scientist and Robin show that multi-agent systems can support hypothesis generation, experimental planning, data analysis, candidate ranking, and iterative refinement of scientific hypotheses.[8,9] Meanwhile, systems for end-to-end AI research automation suggest that agentic AI is beginning to participate in empirical software development and computational research workflows.[10] In chemistry, the role of LLM agents is being redefined: from providers of isolated answers to participants in executable discovery workflows.

However, embedding agents in scientific workflows also exposes a deeper challenge. Scientific discovery depends not merely on whether an agent can call a tool, retrieve information, or generate code, but on whether it can execute a scientifically meaningful process with reliability, transparency, and reproducibility. This issue is acute in computational chemistry, where research workflows span heterogeneous software ecosystems, complex file formats, and high-performance computing (HPC) environments. In this setting, successful execution at the command-line level is not equivalent to scientific validity: an apparently completed workflow may still yield chemically unreasonable structures, numerically unreliable results, or outputs that cannot be reproduced because critical provenance information has been lost.

Recent developments in computational-chemistry agents offer a concrete route toward closing this gap. A representative example is the OpenClaw-based framework for automating computational chemistry workflows, which separates general-purpose agent control from domain-specific scientific execution.[11] In this design, OpenClaw decouples general agent coordination from reusable computational chemistry skills and grounds their execution in real HPC environments. A methane oxidation reactive molecular dynamics (MD) case study illustrates how this architecture supports multistep simulation workflows with monitoring, recovery, and downstream analysis. Related computational-chemistry agents reinforce this

workflow-oriented view. El Agente Q demonstrates how natural-language instructions can be translated into executable quantum-chemistry workflows, while ChemGraph extends agentic automation to molecular simulation tasks such as structure generation, geometry optimization, vibrational analysis, and thermochemistry calculations.[12,13] These examples indicate that computational-chemistry agents must move beyond flexible tool access toward reusable procedures, explicit workflow states, validated execution, and traceable recovery.

Building on these developments and the challenges they reveal, we argue that computational chemistry serves as a key testing ground for the next generation of scientific AI agents. For agents to deliver valid research value in this field, they must preserve scientific correctness across the full workflow — from setting up input files and selecting computational methods to verifying output results and maintaining complete, traceable records of every step. To move beyond basic tool calling toward truly dependable scientific execution, we identify three core requirements. First, as computational chemistry software ecosystems grow larger and more complex, agents must be structured around reusable, task-specific skills. Rather than calling individual tools in isolation, they should package complete, repeatable scientific procedures into modular components that can be flexibly combined. Second, agent frameworks for computational chemistry must strike a proper balance between flexibility and stability: more adaptive than rigid, predefined workflows, yet more consistent and trustworthy than unconstrained LLM agents with no execution boundaries. Third, since LLMs do not natively hold stable, accurate, and verifiable procedural knowledge for specialized computational science tasks, agentic grounding is essential. This mechanism links high-level reasoning and decision-making to real execution logs, runtime error feedback, domain-specific validation rules, full process records, and human supervision.

By redefining scientific agents primarily as reliability-focused execution systems, this Perspective clarifies the practical capabilities that computational chemistry demands from next-generation agentic AI. Through this reorientation, it outlines a clear path toward more rigorous, reproducible, and practically useful agent-driven computational chemistry research.

2. Software Complexity Demands Skill-Based Scientific Agents

The increasing complexity of computational chemistry software ecosystems requires scientific agents to move beyond isolated tool access toward skill-based execution. Modern computational chemistry relies on heterogeneous and continuously evolving software infrastructures, where different programs often use distinct input formats, execution conventions, dependency requirements, and version-specific behaviors. In practice, even a common molecular simulation task is not completed by calling a single tool. For example, preparing a ligand–protein system may fail because the ligand contains nonstandard atoms that cannot be automatically parameterized, or because the generated topology is inconsistent with the coordinate file. Similarly, a quantum-chemistry calculation may terminate not because the method is unavailable, but because the input structure leads to convergence failure and requires expert adjustment of the initial geometry or calculation settings. These examples demonstrate that the practical bottleneck is often not the absence of computational methods, but the difficulty of coordinating tools, data representations, intermediate files, runtime environments, and validation rules across multistep research tasks.

Simply connecting an LLM agent to more tools does not provide this coordination capability. Tool interfaces expose executable operations, but they rarely encode the expert knowledge needed to use those operations correctly. Computational chemistry workflows involve many implicit, experience-dependent decisions: molecular preprocessing, parameter assignment, input validation, convergence diagnosis, failure recovery, and output interpretation. These decisions cannot be fully captured by command-line options or API descriptions alone. The real scientific competence lies in knowing how tools should be combined, under what assumptions they should be used, how intermediate results should be checked, and when execution should be halted or revised.

Skill-based agents provide a more suitable abstraction for this missing layer. A tool

exposes a computational capability; a skill encodes task-specific execution competence. In computational chemistry, a skill should not be understood as a simple script. It should package software-facing operations together with usage instructions, workflow templates, parameter constraints, validation checks, expected outputs, and failure-handling logic. Scientific agents for computational chemistry must therefore move from API-level tool use to workflow-level scientific execution.

The OpenClaw-based computational chemistry framework provides a concrete demonstration of why software complexity demands skill-based agents. Instead of treating computational chemistry automation as a sequence of isolated tool calls, the framework separates general-purpose agent control from software-facing scientific execution. OpenClaw coordinates context, state, skill selection, and execution supervision, while task-planning skills convert natural-language goals into executable workflow specifications and domain skills encapsulate computational chemistry procedures, software interfaces, and validation checks. The DPDispatcher skill further grounds these procedures in heterogeneous HPC environments through job submission, monitoring, and result retrieval. This design shows that, in complex computational chemistry workflows, reusable skills are not merely wrappers around tools; they provide the procedural layer needed to connect reasoning, software execution, runtime feedback, and infrastructure management in a maintainable way.

Future scientific agents will likely accumulate capability through the continuous expansion, testing, maintenance, and evolution of such skill ecosystems. A useful computational chemistry skill library should therefore support reproducible execution, validation, provenance tracking, and iterative improvement from real-world failures. Under this view, the long-term progress of scientific agents will depend not on larger language models or longer tool lists, but on whether expert knowledge can be encoded into reusable, testable, auditable, and evolvable skills.

3. Balancing Flexibility and Stability in Computational Chemistry Agents

Computational chemistry automation calls for frameworks that combine the flexibility of agentic reasoning with the stability of structured workflows. Traditional workflow systems have been essential for reproducible and traceable computation because they explicitly encode task dependencies, execution logic, and data provenance. However, many computational chemistry tasks cannot be fully specified as fixed task graphs before execution. Cross-software adaptation, runtime failures, context-dependent parameter choices, and unexpected intermediate outputs often require active inspection, revision, or bounded recovery during the run.

General LLM agents offer complementary flexibility. They can interpret natural-language objectives, decompose tasks, select tools, generate scripts, and revise plans based on intermediate feedback. Yet this flexibility becomes risky when planning logic, tool routing, recovery policies, and execution assumptions are embedded implicitly in prompts or tightly coupled agent stacks. In such designs, it is difficult to identify where the workflow is represented, where domain constraints are enforced, and how failures are detected and corrected. The result is an automation system that may adapt at runtime but remains difficult to inspect, maintain, reproduce, and validate. A useful computational chemistry agent should therefore occupy a middle ground: it should preserve the agent's capacity for dynamic planning while making workflows, states, intermediate results, and recovery actions explicit. Tools execute commands; skills execute scientific intentions. The key is not to replace workflows with open-ended autonomy, but to make workflows more adaptive, state-aware, and recoverable under scientific constraints.

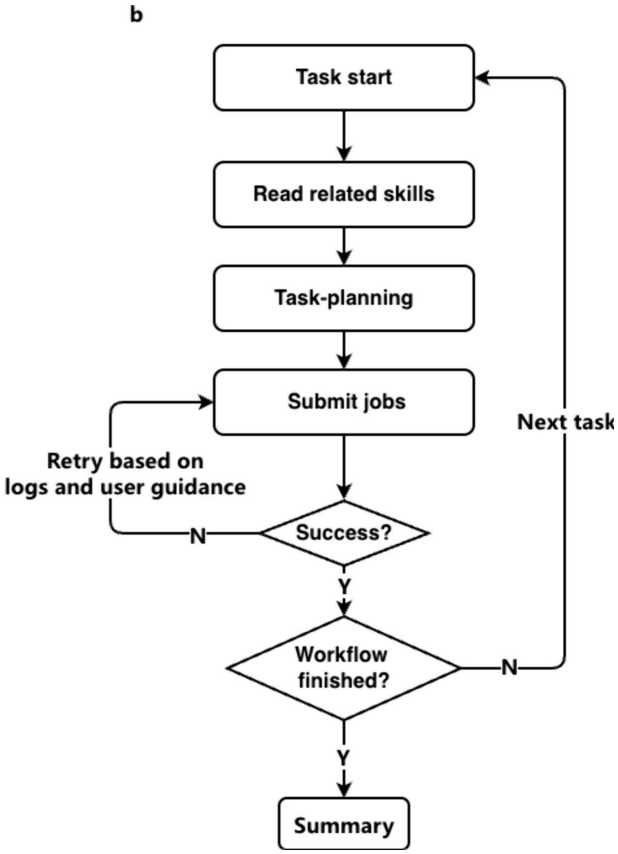

Figure 1: Runtime execution loop of a skill-based computational chemistry agent.

The OpenClaw methane oxidation reactive MD case provides a useful illustration of this balance. As shown in Figure 1, the agent follows a structured execution loop that includes skill retrieval, task planning, job submission, success checking, retry, and final summarization, rather than relying on either a fixed script or unconstrained autonomous actions. In the methane oxidation case, this general loop was instantiated by combining reusable software-facing skills for molecular preparation, file conversion, Deep Potential MD with LAMMPS, scheduler dispatch, and trajectory analysis. The workflow remained flexible because the agent could respond to runtime feedback, but it was also stabilized by explicit workflow states, execution logs, and bounded recovery rules. For example, when a scheduler-submitted LAMMPS job failed due to an environment-activation issue, the system used scheduler and DPDispatcher logs together with the workflow state to revise the execution wrapper and resume the workflow. This example demonstrates how computational chemistry agents can

adapt to runtime conditions while preserving the structured control and recovery semantics needed for reliable execution.

The central design goal, therefore, is not maximal autonomy but operational accountability. For computational chemistry agents, reliability depends on more than the ability to act independently. It requires workflow structures that can be inspected, intermediate evidence that can be preserved and revisited, and recovery actions that remain bounded by predefined scientific and execution rules. At the same time, such systems must retain sufficient flexibility to respond to runtime variability, including software failures, unexpected outputs, and context-dependent decisions. From this perspective, next-generation computational chemistry agents are better understood not as replacements for workflows, but as adaptive extensions of them: they embed agentic reasoning within explicit procedural structures, verifiable execution states, and recoverable scientific operations.

4. Grounding Procedural Scientific Knowledge in Agentic Execution

LLMs can integrate textual knowledge, generate plans, write scripts, and explain results, but they do not inherently possess stable, precise, and verifiable procedural knowledge for computational science. This limitation is especially consequential in computational chemistry, where successful execution requires more than knowing the names of methods or software packages. It depends on executable conventions: valid input formats, parameter settings, software-specific rules, numerical checks, physical constraints, and expert judgment about whether an output is scientifically meaningful. Without grounding in these executable and verifiable elements, an LLM may generate a plausible workflow that fails in practice, or, more dangerously, one that runs successfully while producing unreliable scientific conclusions.

This limitation motivates agentic grounding for procedural scientific knowledge. In computational chemistry, grounding means anchoring an LLM-based agent's reasoning in exe-

cutable tools, domain-specific skills, runtime feedback, validation rules, provenance records, and human oversight. The purpose is not to make the LLM internally store all computational chemistry expertise. Rather, the LLM should operate within a scientific execution system where its plans are continuously checked against real software behavior, explicit task specifications, intermediate outputs, error messages, and domain validation criteria.

The OpenClaw computational chemistry framework illustrates this principle by externalizing procedural knowledge into skills rather than leaving it implicit in the model. Planning skills translate broad scientific goals into executable task specifications, while domain skills encode software-facing procedures, usage constraints, templates, and validation logic. In this architecture, the LLM reasons over the user goal, loaded skills, current state, and execution feedback, but its actions are grounded by the procedures and evidence returned from real computational tools. Runtime artifacts such as logs, return codes, generated files, scheduler states, and schema checks therefore become part of the agent's decision-making context.

This grounding also clarifies the boundary of scientific agency. A computational chemistry agent can coordinate software execution, maintain workflow state, recover from certain operational failures, and produce traceable outputs. However, it does not replace expert scientific validation. Human researchers must still choose appropriate physical models, inspect input files, evaluate numerical protocols, and judge whether final observables support a scientific conclusion. Reliable agents should therefore be understood not as autonomous scientific authorities, but as grounded execution systems that connect LLM reasoning with executable procedures, software feedback, validation mechanisms, and expert supervision.

This perspective shifts the design goal of scientific agents. The central challenge is not to make LLMs memorize more procedural knowledge, but to build systems in which such knowledge is externalized, tested, updated, and enforced through the interaction between agents, skills, tools, environments, and scientists. For computational chemistry, agentic grounding is therefore not an optional safeguard; it is the condition that allows language-based reasoning to become scientifically usable execution.

Outlook and Conclusion

The transition from tool-using language models to reliable scientific agents for computational chemistry should not be understood as a simple expansion of tool access. The central issue is whether language-based reasoning can be transformed into chemically meaningful, traceable, and recoverable execution. In this Perspective, we have argued that this transition depends on three connected requirements: skill-based execution for complex software ecosystems, agent frameworks that balance flexibility with stability, and agentic grounding of procedural scientific knowledge in tools, feedback, validation, provenance, and expert oversight.

Several challenges will determine whether such systems become practically useful. First, computational chemistry skills must evolve from ad hoc wrappers into reusable and testable scientific modules, with clear assumptions, applicability boundaries, dependencies, validation criteria, and failure records. Second, agent workflows need stronger provenance and audit mechanisms so that task specifications, intermediate files, software environments, execution traces, recovery actions, and final outputs can be inspected and reproduced. Third, as agents gain access to HPC resources, databases, files, and eventually experimental platforms, safety mechanisms such as permission control, sandboxed execution, bounded recovery, and human-in-the-loop supervision will become essential. These requirements are not peripheral engineering details; they are integral to making agentic execution scientifically accountable.

Looking ahead, the goal should not be to build agents that replace workflows or act as autonomous scientific authorities. Instead, computational chemistry agents should become adaptive extensions of scientific workflows: systems that externalize procedural knowledge into maintainable skills, ground reasoning in executable evidence, and preserve expert control over scientific judgment. The future of agentic computational chemistry will therefore be defined less by unrestricted autonomy than by reliable coordination among models, skills, tools, computational environments, and human scientists. Only when these elements are integrated into reproducible and verifiable execution loops can scientific agents become trustworthy infrastructure for computational chemistry and molecular discovery.

Acknowledgement

acknowledgement

References

(1) Li, X. A Review of Prominent Paradigms for LLM-Based Agents: Tool Use, Planning (Including RAG), and Feedback Learning. Proceedings of the 31st International Conference on Computational Linguistics. pp 9760–9779.

(2) Xu, R.; Yan, Y. Agent Skills for Large Language Models: Architecture, Acquisition, Security, and the Path Forward. http://arxiv.org/abs/2602.12430.

(3) Wang, C.; Yu, Z.; Xie, X.; Yao, W.; Fang, R.; Qiao, S.; Cao, K.; Zheng, G.; Qi, X.; Zhang, P.; Deng, S. SkillX: Automatically Constructing Skill Knowledge Bases for Agents. http://arxiv.org/abs/2604.04804.

(4) Skills · OpenClaw. https://docs.openclaw.ai/tools/skills.

(5) M. Bran, A.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *6*, 525–535.

(6) Ramos, M. C.; Collison, C. J.; White, A. D. A Review of Large Language Models and Autonomous Agents in Chemistry. *16*, 2514–2572.

(7) Xie, Z.; Luo, M.; Ye, Z.; Chen, L.; Zhu, Z.; Jiang, J. Autonomous Chemistry and Materials Innovation Driven by Scientific Agents. *6*, 2659–2669.

(8) Ghareeb, A. E.; Chang, B.; Mitchener, L.; Yiu, A.; Szostkiewicz, C. J.; Shved, D.; Gyimesi, G. J.; Laurent, J. M.; Wright, S. M.; Razzak, M. T.; White, A. D.; Finnemann, S. C.; Hinks, M. M.; Rodriques, S. G. A Multi-Agent System for Automating Scientific Discovery. 1–3.

(9) Gottweis, J. et al. Accelerating Scientific Discovery with Co-Scientist. 1–3.

(10) Lu, C.; Lu, C.; Lange, R. T.; Yamada, Y.; Hu, S.; Foerster, J.; Ha, D.; Clune, J. Towards End-to-End Automation of AI Research. *651*, 914–919.

(11) Ding, M.; Huang, C.; Hu, Y.; Li, Y.; Lu, Z.; Yu, X.; Zhang, D.; Zhai, W.; Zhu, T.; Gu, Q.; Zeng, J. Automating Computational Chemistry Workflows via OpenClaw and Domain-Specific Skills. *22*, 5919–5929.

(12) Zou, Y. et al. El Agente: An Autonomous Agent for Quantum Chemistry. *8*.

(13) Pham, T. D.; Tanikanti, A.; Keçeli, M. ChemGraph as an Agentic Framework for Computational Chemistry Workflows. *9*, 33.